KNOWLEDGE BYTES

Codebase Understanding Guide

AWS Serverless Ticketing System • Supportly

Date generated: 2026-10-13

Documentation only. No application source code was modified.

1. Project Overview

Supportly is a serverless help-desk ticketing system. Customers create tickets, upload attachments, and comment; Agents triage the queue, update status/assignee, and reply. Frontend: Vue 3 + Vite + Tailwind SPA with Cognito Hosted UI (OIDC code flow via oidc-client-ts). Backend: Express apps deployed two ways — (a) local mock server with JSON-file persistence, (b) production Express on Lambda (serverless-http) backed by DynamoDB + S3 presigned URLs, fronted by API Gateway HTTP API with a Cognito JWT authorizer. Seven ticket routes are identical in both paths.

Verified from inspected code: local auth uses deterministic mock tokens (mock.<b64>.local) verified against fixtures in backend/src/lib/mockUsers.js; production auth reads verified JWT claims (cognito:groups) and never trusts frontend-sent roles; customers are scoped to their own tickets and non-owners receive 404 (never 403) to avoid existence leaks; only Agents may PUT tickets; validation/filtering/scoping helpers are pure shared functions; presigned uploads are user-namespaced with 5 MB / MIME allowlist; the single-table module (tkt-tickets, pk/sk) is implemented and unit-tested but handlers point at it while the table itself is not yet provisioned (Stage 3 approval pending) — deployment JSONs are plans marked NOT APPLIED/CREATED. AWS deployment is documented as BLOCKED; local suite (backend 83/83, frontend 9/9, 14-step smoke) is the verified green path.

Inferred but not executed here: production DynamoDB/S3/Cognito behavior (code is unit-tested with injected fakes, never run against AWS in this task — no AWS calls were made).

2. Codebase Map

backend/server.js — local runnable API (mock auth + JSON file + local disk uploads). backend/aws-server.js — production Express (DynamoDB two-table) wrapped by backend/lambda.js via serverless-http. backend/src/lib/ — auth.js (claims/authz), mockUsers.js (local fixtures), validation.js (pure rules), response.js (error envelope), uploads.js (presign validation), tickets.js (two-table DAL), ticketsSingleTable.js (required tkt-tickets single-table DAL, 351 lines). backend/src/handlers/ — seven tkt-* Lambda entry points (one per route). backend/scripts/ — smoke-local, reset-local-data, verify-live. backend/deploy/ — functions.json, api-routes.json and other Stage plans (not applied). frontend/src/ — main.js/router/auth/services/views; auth/ (cognito.js, mock.js, guard.js, useAuth.js); views (Landing, Login, CustomerLogin, AgentLogin, Callback, Dashboard=customer, AgentDashboard, HelloWorld). frontend/tests/, backend/tests/ — node:test suites. Root scripts — seed-ticket-batch.ps1, scripts/configure-frontend-aws.ps1. Excluded: node_modules, dist/, *.zip, backup dirs, *.before-* snapshots, package-lock.json, seeded-ticket dumps, smoke-response JSONs (see appendix).

3. KNOWLEDGE BYTES — Backend entry & runtime

File: backend/package.json

Byte 1: Scripts are the project's control panel

Builds on: None — starting point

In plain terms: This file declares how to run, test, reset, and smoke-test the backend. Think of it as the button panel for developers.

The code:

```
{
  "start": "node server.js",
  "test": "node --test tests/validation.test.js tests/auth.test.js ...",
  "reset:demo": "node scripts/reset-local-data.js",
  "smoke": "node scripts/smoke-local.js"
}
```

What's happening: Each script maps a npm command to a file: start boots the local mock API, test runs eight test files with the Node built-in runner, reset:demo wipes only local data, smoke runs the 14-step localhost check, verify:live is a safe read-only live probe.

Why it matters: Without this map you would not know which file boots what; it also proves tests need no external runner.

Byte 2: Dependencies reveal the architecture

Builds on: Byte 1

In plain terms: The dependency list tells you this backend is Express plus the AWS SDK v3, with nothing else doing magic.

The code:

```
"dependencies": {
  "express": "^5.3.0",
  "serverless-http": "^4.0.0",
  "@aws-sdk/client-dynamodb": "^3.1149.0",
  "@aws-sdk/client-s3": "^3.1149.0"
}
```

What's happening: Express serves HTTP; serverless-http adapts Express to Lambda; lib-dynamodb is the DynamoDB document client; client-s3 + presigner mint upload/download URLs. cors/dotenv handle browser CORS and env files.

Why it matters: If S3 or DynamoDB code looks unfamiliar, this list tells you exactly which SDK packages to read about.

PUTTING IT TOGETHER. package.json shows a plain Express service whose AWS abilities come from four SDK packages, driven by npm scripts for local run, test, reset, and smoke — the same scripts the README's 2-minute quickstart uses.

File: backend/server.js (local mock API, 399 lines)

Byte 1: Local server mirrors production paths exactly

Builds on: None — starting point

In plain terms: This is the runnable local twin of the production stack: same 7 ticket routes and same 401/403/404 rules, but with fake logins instead of Cognito.

The code:

```
app.post('/tickets', requireMockAuth, (req, res) => {...})
app.get('/tickets', requireMockAuth, ...)
app.put('/tickets/:ticketId', requireMockAuth, ...)
```

What's happening: Seven routes keep their exact production paths so the frontend cannot tell local from prod. Every ticket route sits behind requireMockAuth; only the /auth/mock-* and /mock-uploads/* helpers are local-only.

Why it matters: Frontend developers can build the full UI without any AWS account, yet authorization behavior stays faithful.

Byte 2: Identity comes only from the verified token

Builds on: Byte 1

In plain terms: The server never believes a user id typed into a form — it reads who you are from your login token.

The code:

```
const match = /^Bearer (.+)$/i.exec(header);
const user = match ? verifyMockToken(match[1].trim()) : null;
if (!user) return sendError(res, 401, 'UNAUTHORIZED', ...);
```

What's happening: requireMockAuth extracts the Bearer token and looks it up in the fixture table; role is re-read from the fixture, so editing the token cannot escalate privilege. createdBy is always req.user.sub.

Why it matters: This kills an entire class of spoofing bugs where a customer could claim to be someone else.

Byte 3: JSON file stands in for DynamoDB

Builds on: Byte 1

In plain terms: Tickets live in a plain JSON file on disk, shaped like the real database rows so code stays portable.

The code:

```
function loadTickets() {
  const parsed = JSON.parse(fs.readFileSync(DATA_FILE, 'utf8'));
  return parsed.map((t) => ({ ...t,
    activity: Array.isArray(t.activity) ? t.activity : [],
    comments: Array.isArray(t.comments) ? t.comments : [], ... }));
}
```

What's happening: loadTickets normalizes missing arrays (activity/comments/attachments) so old seed files never crash new code; saveTickets rewrites the whole file. The documented mapping lives in docs/ARCHITECTURE.md.

Why it matters: Persistence survives restarts with zero infrastructure, and the shape parity keeps the local/prod switch honest.

Byte 4: Customers get 404, never a leak

Builds on: Bytes 1–2

In plain terms: When you ask for someone else's ticket, the server pretends it does not exist rather than saying forbidden.

The code:

```
const ticket = findTicket(loadTickets(), req.params.ticketId);
if (!ticket || !canViewTicket(ticket, req.user))
  return sendError(res, 404, 'NOT_FOUND', 'Ticket not found.');
```

What's happening: canViewTicket lets Agents see everything but restricts customers to their own createdBy. Merging 'missing' and 'invisible' into one 404 prevents user-enumeration attacks.

Why it matters: Remove this and customer-b could probe which ticket ids exist by watching 403 vs 404.

Byte 5: Uploads are local files behind a mock presigned URL

Builds on: Bytes 1–3

In plain terms: File upload locally reserves a namespaced key and hands the browser a local PUT address — clearly flagged mock:true so nobody mistakes it for S3.

The code:

```
const key = `local/${req.user.sub}/${randomUUID()}-${filename}`;
return res.json({ uploadUrl: `${base}/mock-uploads/${key}`, key, expiresIn: 300, mock: true });
```

What's happening: keyToSafePath rejects traversal (..), enforces the local/<owner>/ prefix, and checks the owner on PUT; download checks owner-or-Agent. MIME and 5 MB limits are enforced before bytes hit disk.

Why it matters: Attachments work end-to-end offline while the key-namespace discipline previews the S3 layout.

PUTTING IT TOGETHER. server.js is a faithful local double: identical routes and authz outcomes, token-derived identity, JSON-file persistence mirroring the DynamoDB shape, 404-masked authorization, and disk-backed mock uploads — everything the UI needs with no cloud.

File: backend/aws-server.js (production Express, 372 lines)

Byte 1: Same routes, real DynamoDB behind them

Builds on: None — starting point

In plain terms: This is the deployable twin of server.js: identical route semantics, but every read/write goes to DynamoDB tables.

The code:

```
const TICKETS_TABLE = process.env.TICKETS_TABLE || 'Tickets';
const COMMENTS_TABLE = process.env.COMMENTS_TABLE || 'TicketComments';
app.use('/tickets', attachUser({ required: true }));
```

What's happening: Table names come from env with legacy defaults; attachUser({required:true}) rejects any request without verified claims (401). Route handlers for create/list/get/update/comment are one-to-one with server.js.

Why it matters: Operators can reason about local vs prod as one behavior with two storage engines.

Byte 2: Comments live in their own table, ordered by query

Builds on: Byte 1

In plain terms: Instead of embedding comments, production stores them as separate rows fetched oldest-first.

The code:

```
const result = await db.send(new QueryCommand({
  TableName: COMMENTS_TABLE,
  KeyConditionExpression: 'ticketId = :id', ... ScanIndexForward: true }));
```

What's happening: Query on the ticketId partition key with ScanIndexForward:true returns chronological comments without sorting in code. getTicketWithComments joins ticket + comments per request.

Why it matters: This is the retained two-table schema (Tickets + TicketComments) documented in tickets.js and ARCHITECTURE.md.

Byte 3: Presigned S3 route is deliberately stubbed to 501

Builds on: Bytes 1–2

In plain terms: The upload endpoint validates input but refuses to sign anything until the bucket is approved — returning 'not configured' on purpose.

The code:

```
if (!process.env.ATTACHMENTS_BUCKET) {
  return createError(res, 501, 'AWS_NOT_CONFIGURED', 'S3 uploads have not been configured
yet.');
}
```

What's happening: Validation runs first so unit tests never touch AWS or incur charges; live getSignedUrl wiring is documented as Stage C work gated on bucket approval.

Why it matters: This honesty prevents half-configured deployments from issuing broken URLs.

PUTTING IT TOGETHER. aws-server.js proves the production path: JWT-gated routes, two-table DynamoDB access with chronological comment queries, and a stubbed upload route that fails safe (501) until S3 is approved — behavior-identical to local except for storage.

File: backend/lambda.js

Byte 1: Five lines bridge Express to Lambda

Builds on: None — starting point

In plain terms: This tiny file turns the whole Express app into a Lambda handler with one library call.

The code:

```
const serverless = require('serverless-http');
const app = require('./aws-server');
module.exports.handler = serverless(app);
```

What's happening: serverless-http translates API Gateway proxy events into Express requests and back, so no route code changes for Lambda. The deployed handler name matches deploy/functions.json entries.

Why it matters: One artifact (ticketing-handlers.zip) can serve all seven functions with only the Handler field differing.

PUTTING IT TOGETHER. lambda.js is pure glue: aws-server plus serverless-http equals a Lambda-ready handler, which is why deployment config can share one zip across seven functions.

4. KNOWLEDGE BYTES — Backend shared libraries

File: backend/src/lib/auth.js (200 lines)

Byte 1: Never verify signatures here — the authorizer does

Builds on: None — starting point

In plain terms: These helpers only read claims the API Gateway JWT authorizer already verified; doing crypto here would create false confidence.

The code:

```
// These helpers NEVER verify signatures themselves — they only parse claims
// that the authorizer has already verified
```

What's happening: getClaimsFromRequest prefers requestContext.authorizer.jwt.claims (verified upstream), falling back to decoding the Bearer payload for local tests only. Production must keep the authorizer.

Why it matters: If this rule broke, anyone could forge a token and the backend would believe it.

Byte 2: Group claim arrives in four shapes — normalize all

Builds on: Byte 1

In plain terms: Cognito groups can look like an array, a JSON string, '[Agent]', or 'Customer,Agent' depending on who flattened them.

The code:

```
function normalizeGroupClaim(raw) {
  if (Array.isArray(raw)) return raw.map(...);
  try { const parsed = JSON.parse(text);
    if (Array.isArray(parsed)) return parsed...; } catch {}
  const unwrapped = text.startsWith('[') && text.endsWith(']') ? text.slice(1,-1) : text;
  return unwrapped.split(',').map((g) => g.trim().replace(...)).filter(Boolean);
}
```

What's happening: The function tries array, then JSON parse, then bracket-strip + comma-split, trimming quotes. Both backend and SPA use the same Agent-membership rule.

Why it matters: Without this, an Agent could be misclassified as Customer on some event shapes and lose access.

Byte 3: Role comes from groups, mock tokens rejected

Builds on: Bytes 1–2

In plain terms: Your role is derived from cognito:groups, and local fake tokens are explicitly refused here.

The code:

```
if (claims.mock === true) return null;
const groups = normalizeGroupClaim(claims['cognito:groups']);
const role = groups.includes(AGENT_GROUP) ? 'Agent' : 'Customer';
```

What's happening: getUserFromClaims needs a sub/username; email may be null. Anyone not in the Agent group defaults to Customer — frontend and backend agree on this rule.

Why it matters: Structural mock rejection is what makes mock tokens worthless in production even if pasted into a header.

Byte 4: Owner-or-Agent checks power every route

Builds on: Byte 3

In plain terms: Two one-line predicates decide who sees tickets and comments throughout the app.

The code:

```
function canViewTicket(ticket, user) {
  if (!ticket || !user) return false;
  if (isAgent(user)) return true;
  const owner = ticket.createdBy || ticket.ownerId;
  return owner === user.sub || (user.email !== null && owner === user.email);
}
```

What's happening: Legacy ownerId is still honored; email fallback covers older rows. canAccessComments delegates to canViewTicket; canUpdateTicket/requireAgent gate PUT to Agents.

Why it matters: Centralizing this keeps all seven routes and both servers consistent.

PUTTING IT TOGETHER. auth.js is the trust boundary: authorizer-verified claims in, normalized user out, with group-shape tolerance, mock rejection, and owner-or-Agent predicates reused everywhere.

File: backend/src/lib/mockUsers.js (77 lines)

Byte 1: Three passwordless fixtures run the whole demo

Builds on: None — starting point

In plain terms: Three hard-coded users — two customers and one agent — let the app run end-to-end with no Cognito.

The code:

```
const MOCK_USERS = [
  { sub: 'customer-a', email: 'customer-a@example.com', name: 'Customer A', role: 'Customer',
groups: [] },
  { sub: 'customer-b', email: 'customer-b@example.com', ... },
  { sub: 'agent-1', email: 'agent-1@example.com', name: 'Agent One', role: 'Agent', groups:
['Agent'] }
];
```

What's happening: No passwords or secrets exist; the login screen lists them openly. customer-b exists specifically to prove isolation (cannot see A's tickets).

Why it matters: Deterministic identities make authz tests repeatable.

Byte 2: Tokens are opaque locally, useless remotely

Builds on: Byte 1

In plain terms: A mock token is just base64 JSON wrapped in mock…..local with a 12-hour expiry.

The code:

```
const payload = { sub: fixture.sub, mock: true, iat: nowSec, exp: nowSec + MOCK_TOKEN_TTL_SEC };
const b64 = Buffer.from(JSON.stringify(payload)).toString('base64url');
return `mock.${b64}.local`;
```

What's happening: verifyMockToken checks shape, mock:true flag, expiry, and fixture existence — then re-reads role from the fixture, ignoring any role claim inside the token.

Why it matters: Token tampering cannot escalate privilege (proven by tests); production rejects mock:true outright.

PUTTING IT TOGETHER. mockUsers.js gives local development three fixed identities and short-lived tokens whose roles always come from the server-side fixture table — convenient locally, inert in production by construction.

File: backend/src/lib/validation.js (182 lines)

Byte 1: Pure functions shared by both servers

Builds on: None — starting point

In plain terms: All validation lives in dependency-free pure functions so local and prod behave identically and tests need no network.

The code:

```
const STATUSES = ['Open', 'In Progress', 'Resolved', 'Closed'];
const PRIORITIES = ['Low', 'Medium', 'High'];
const CATEGORIES = ['Technical Issue', 'Access & Accounts', 'Billing', ...];
```

What's happening: Title ≤120, description ≤5000, comment ≤2000 chars; priority defaults to Medium; category defaults to Technical Issue. Update requires at least one of status/assignee/priority.

Why it matters: One rulebook prevents the classic bug where UI and API disagree on what is valid.

Byte 2: Filtering and scoping are server-side, not UI hints

Builds on: Byte 1

In plain terms: Agents see everything; customers see only their own — enforced in code, not hidden in the UI.

The code:

```
function scopeTicketsToUser(tickets, user) {
  if (!user) return [];
  if (user.role === 'Agent') return tickets;
  return tickets.filter((t) => (t.createdBy || t.ownerId) === user.sub || ... === user.email);
}
```

What's happening: filterTickets applies status/priority/text search then sorts newest-first. Because scoping happens before filtering, customers cannot search across others' tickets.

Why it matters: UI hiding is cosmetic; this is the real boundary.

Byte 3: Attachment keys must name your own uploads

Builds on: Bytes 1–2

In plain terms: When creating a ticket you can attach at most 5 keys, each under your own namespace.

The code:

```
const okPrefix = allowedPrefixes.some((p) => key.startsWith(p));
if (!okPrefix || !rest || rest.includes('..') || owner !== String(userSub)) {
  errors.push('Attachment keys must reference your own uploads.');
```

What's happening: The owner segment (second path part) must equal your sub; traversal (..) and over-long keys are rejected. Local allows local/, prod requires attachments/.

Why it matters: Customers cannot claim each other's files by guessing keys.

PUTTING IT TOGETHER. validation.js is the shared contract: enums and length limits, server-side ownership scoping plus search/filter, and namespaced attachment keys — identical for both servers.

File: backend/src/lib/response.js (66 lines)

Byte 1: One error shape everywhere

Builds on: None — starting point

In plain terms: Every error looks like { error: { code, message } } whether it comes from Express or Lambda.

The code:

```
function errorBody(code, message) {
  return { error: { code, message } };
}
function sendError(res, status, code, message) {
  return res.status(status).json(errorBody(code, message));
}
```

What's happening: sendError/sendOk serve Express; httpError throws coded errors for Lambda; toLambdaResponse converts them to proxy responses. Frontend api.js reads data?.error?.message accordingly.

Why it matters: Consistent envelopes let the UI show server messages without per-route parsing.

Byte 2: 500s never leak internals; bad JSON is a 400

Builds on: Byte 1

In plain terms: Crashes collapse to a generic message, while malformed request bodies are client errors, not server faults.

The code:

```
statusCode === 500
  ? errorBody('INTERNAL_ERROR', 'An unexpected server error occurred.')
  : errorBody(codeOf(errOrBody), errOrBody.message);
function parseJsonBody(event) {
  try { return JSON.parse(event.body); }
  catch { throw httpError(400, 'VALIDATION_ERROR', 'Request body must be valid JSON.'); }
```

What's happening: statusOf/codeOf default missing fields safely. Stack traces never reach the browser.

Why it matters: Prevents information disclosure and misclassified 500s that would page on-call for a client typo.

PUTTING IT TOGETHER. response.js keeps the API's promise: uniform {error:{code,message}} envelopes, safe 500s, and 400-on-bad-JSON — the contract the frontend error UI depends on.

File: backend/src/lib/uploads.js (82 lines)

Byte 1: Validation without AWS calls

Builds on: None — starting point

In plain terms: Upload rules are pure functions so tests never touch S3 or spend money.

The code:

```
const MAX_FILE_BYTES = 5 * 1024 * 1024;
const ALLOWED_MIME = new
Set(['image/png','image/jpeg','image/gif','image/webp','application/pdf','text/plain']);
```

What's happening: validatePresignedRequest requires filename + allowlisted MIME, optional fileSize within 1–5 MB. sanitizeFilename strips paths and unsafe chars (→ underscores, ≤180 chars).

Why it matters: The live getSignedUrl call lives elsewhere and runs only after bucket approval.

Byte 2: Keys are namespaced per user; display names strip UUIDs

Builds on: Byte 1

In plain terms: Each upload key embeds who owns it plus a random id, while the UI shows the original filename.

The code:

```
return `attachments/${safeSub}/${scope}${uuid}-${safeFile}`;
function displayNameFromKey(stored) {
  const m = /^[0-9a-f]{8}-...-(.+)$/i.exec(stored);
  return m ? m[1] : stored;
}
```

What's happening: buildUploadKey sanitizes the sub and filename; ticketId optionally scopes further. displayNameFromKey removes the uuid- prefix for human display.

Why it matters: Namespacing plus validation.js key checks is what stops cross-user file claims.

PUTTING IT TOGETHER. uploads.js plans safe uploads offline: size/MIME/filename checks, per-user namespaced keys, and display-name recovery — signing itself is deferred to the handler that owns AWS credentials.

File: backend/src/lib/tickets.js (two-table DAL, 203 lines)

Byte 1: Injected db keeps AWS out of tests

Builds on: None — starting point

In plain terms: Every function takes a db object with a send(cmd) method, so tests pass a fake and production passes the real client.

The code:

```
async function getTicketItem(db, tables, ticketId) {
  const out = await db.send(new GetCommand({ TableName: tables.tickets, Key: { ticketId } }));
```

```
  return out.Item || null;
}
```

What's happening: tablesFromEnv reads TICKETS_TABLE/COMMENTS_TABLE with legacy defaults. Coded failures use httpError for uniform Lambda responses.

Why it matters: Architecture becomes testable without mocks of the network.

Byte 2: Missing-or-invisible both yield 404

Builds on: Byte 1

In plain terms: A helper refuses to tell callers whether a ticket is absent or merely not theirs.

The code:

```
async function requireVisibleTicket(db, tables, user, ticketId) {
  const ticket = await getTicketItem(db, tables, ticketId);
  if (!ticket || !canViewTicket(ticket, user))
    throw httpError(404, 'NOT_FOUND', 'Ticket not found.');
```

What's happening: create/list/update/comment functions layer validation → visibility → write, appending activity entries (CREATED/STATUS_CHANGED/ASSIGNED/COMMENT_ADDED) with actorId and timestamps.

Why it matters: This single helper is the anti-enumeration guarantee for the two-table path.

PUTTING IT TOGETHER. tickets.js implements the retained two-table model (Tickets + TicketComments) with injectable persistence, 404-masked visibility, and activity journaling — the DAL that aws-server.js mirrors inline.

File: backend/src/lib/ticketsSingleTable.js (required tkt-tickets DAL, 351 lines)

Byte 1: One table, three item kinds by sort key

Builds on: None — starting point

In plain terms: The building-plan schema packs ticket, activity, and comments into one DynamoDB table distinguished by sk prefixes.

The code:

```
// { pk: TICKET#<id>, sk: META }                  ticket attributes
// { pk: TICKET#<id>, sk: ACTIVITY#<time>#<uuid>} activity entries
// { pk: TICKET#<id>, sk: COMMENT#<time>#<uuid> } comment entries
```

What's happening: pkFor builds TICKET#<id>; queryItems fetches COMMENT#/ACTIVITY# by begins_with with ScanIndexForward ordering. toPublicTicket reassembles meta+comments+activity into the API shape.

Why it matters: Single-table design removes the second table and makes per-ticket reads one partition query.

Byte 2: Deployment contract names with safe aliases

Builds on: Byte 1

In plain terms: Table and bucket env names are pinned, with backward-compatible fallbacks that never point at legacy tables.

The code:

```
function tableFromEnv(env = process.env) {
  return { table: env.TABLE_NAME || env.TICKETS_TABLE || env.TKT_TABLE || REQUIRED_TABLE };
}
const REQUIRED_TABLE = 'tkt-tickets';
```

What's happening: attachmentsBucket accepts BUCKET_NAME or ATTACHMENTS_BUCKET. Signatures match tickets.js so handlers rewire by import swap.

Why it matters: Operators cannot accidentally deploy against the old tables via a stale env var.

Byte 3: Download signing degrades gracefully to null

Builds on: Bytes 1–2

In plain terms: Attachment download URLs are minted only when a bucket exists; otherwise the API returns keys plus display names.

The code:

```
async function defaultGetSigner(key) {
  const bucket = attachmentsBucket();
  if (!bucket) return null;
  const { S3Client, GetObjectCommand } = require('@aws-sdk/client-s3');
  ... return getSignedUrl(client, new GetObjectCommand({ Bucket: bucket, Key: key }), {
expiresIn: 300 });
}
```

What's happening: Lazy-requiring the S3 SDK inside the function keeps unit tests AWS-free. 300-second expiry matches the upload URL lifetime.

Why it matters: The ticket detail page works before and after S3 approval, with real download links appearing once configured.

PUTTING IT TOGETHER. ticketsSingleTable.js is the forward-looking DAL: single tkt-tickets table with META/ACTIVITY#/COMMENT# items, contract-pinned env names, graceful no-bucket degradation, and handler-compatible signatures — implemented and tested, awaiting table approval.

5. KNOWLEDGE BYTES — Lambda handlers (tkt-*)

File: backend/src/handlers/createTicket.js + listTickets.js

Byte 1: Thin handlers: auth, delegate, respond

Builds on: None — starting point

In plain terms: Each Lambda file does only three things — prove who you are, call the DAL, format the reply.

The code:

```
const user = getUserFromLambdaEvent(event);
if (!user) throw httpError(401, 'UNAUTHORIZED', 'Valid authentication is required.');
const ticket = await createTicketItem(db, tableFromEnv(), user, parseJsonBody(event));
return toLambdaResponse({ ticket }, 201);
```

What's happening: buildHandler(db, signer?) injects fakes for tests; module exports both handler (wired to real DynamoDB in ap-southeast-2) and buildHandler. try/catch converts any coded error via toLambdaResponse.

Why it matters: Business logic stays in lib/ so handlers are auditable at a glance.

Byte 2: List honors query filters customers already know

Builds on: Byte 1

In plain terms: Listing passes ?status=&priority=&q= straight to the shared filter after scoping.

The code:

```
const result = await listTicketItems(db, tableFromEnv(), user, event?.queryStringParameters ||
{}, urlSigner);
```

What's happening: Agents get all tickets; customers get only theirs (scopeTicketsToUser inside the DAL). The urlSigner enriches attachments with download URLs when S3 exists.

Why it matters: Search UX and API semantics cannot drift apart.

PUTTING IT TOGETHER. create/list handlers show the pattern all seven follow: JWT claims → DAL call on tkt-tickets → uniform Lambda response, with 201 on create and scoped filtered lists.

File: backend/src/handlers/getTicket.js + listComments.js

Byte 1: Detail and comments share the visibility gate

Builds on: None — starting point

In plain terms: Both read paths funnel through requireVisibleTicket so owner-or-Agent is checked identically.

The code:

```
const meta = await requireVisibleTicket(db, tables, user, event?.pathParameters?.ticketId);
return toLambdaResponse({ ticket: await withComments(db, tables, meta, urlSigner) }, 200);
```

What's happening: listComments then lists COMMENT# items oldest-first; getTicket returns meta + comments + activity + attachment URLs. ticketId comes from pathParameters.

Why it matters: One gate means no route forgets the 404-masking rule.

PUTTING IT TOGETHER. Get-ticket and list-comments are read-only siblings: same gate, same 404-on-invisible behavior, chronological comments — safe to expose to both roles.

File: backend/src/handlers/updateTicket.js + addComment.js + presignedUrl.js

Byte 1: Only Agents may mutate tickets

Builds on: None — starting point

In plain terms: Status/assignee/priority changes reject non-Agents inside the DAL with 403 (or 401 when anonymous).

The code:

```
if (!user || user.role !== 'Agent') {
  throw httpError(user ? 403 : 401, user ? 'FORBIDDEN' : 'UNAUTHORIZED', ...); }
```

What's happening: updateTicketItem appends STATUS_CHANGED/ASSIGNED activity with actorId; assignee accepts assignee or assigneeId keys (pickAssignee). Customers attempting PUT get FORBIDDEN.

Why it matters: Agent-only mutation is the core support-desk invariant.

Byte 2: Comments stamp authorship from claims

Builds on: Byte 1

In plain terms: Comment author and role come from the verified token, never the request body.

The code:

```
const comment = await addCommentItem(db, tableFromEnv(), user, event?.pathParameters?.ticketId,
parseJsonBody(event));
return toLambdaResponse({ comment }, 201);
```

What's happening: addCommentItem validates text (≤2000), checks visibility, writes the COMMENT# item, and appends COMMENT_ADDED activity + updatedAt.

Why it matters: Spoofed author names are impossible.

Byte 3: Upload URLs are short-lived and bucket-gated

Builds on: Bytes 1–2

In plain terms: The presign handler validates, plans a namespaced key, then signs only if a bucket is configured.

The code:

```
const plan = planPresignedUpload(user, parseJsonBody(event));
const bucket = process.env.BUCKET_NAME || process.env.ATTACHMENTS_BUCKET;
if (!bucket) throw httpError(501, 'AWS_NOT_CONFIGURED', ...);
const uploadUrl = await signer({ bucket, key: plan.key, contentType: plan.contentType });
```

What's happening: UPLOAD_EXPIRY_SECONDS = 300; signer defaults to S3 PutObject presigning but is injectable for tests. 501 (not 500) signals 'approved later', matching aws-server.js.

Why it matters: No bucket, no signed URL, no charge, no broken client state.

PUTTING IT TOGETHER. Mutation handlers complete the set: Agent-only updates with activity audit, claim-stamped comments, and bucket-gated 5-minute upload URLs — each a thin wrapper over a tested lib function.

6. KNOWLEDGE BYTES — Scripts, services, data, deploy plans

File: backend/scripts/smoke-local.js + reset-local-data.js + verify-live.js

Byte 1: Smoke test uses throwaway data by default

Builds on: None — starting point

In plain terms: The 14-step localhost check spins the real server against temp files, then deletes them unless KEEP_DATA=1.

The code:

```
tmpRoot = fs.mkdtempSync(path.join(os.tmpdir(), 'ticketing-smoke-'));
process.env.DATA_FILE = path.join(tmpRoot, 'tickets.json');
// exercises all 7 routes + auth + attachments ... prints PASS/FAIL per step
```

What's happening: api() helper wraps fetch with optional Bearer tokens; check() counts failures and exits nonzero. Covers 401/403/404, role spoof rejection, upload/download persistence.

Why it matters: Repeatable green runs without polluting demo data.

Byte 2: Reset touches only local files, never AWS

Builds on: Byte 1

In plain terms: Resetting the demo rewrites tickets.json to [] and recreates uploads/ — nothing else.

The code:

```
fs.writeFileSync(dataFile, '[]');
if (fs.existsSync(uploadDir)) fs.rmSync(uploadDir, { recursive: true, force: true });
```

What's happening: Paths honor DATA_FILE/UPLOAD_DIR overrides. The header comment states 'Never touches AWS' — safe to run anytime.

Why it matters: A one-command clean slate for demos and manual testing.

Byte 3: Live verifier is read-only and skips without a URL

Builds on: Bytes 1–2

In plain terms: Against a deployed API it only GETs /health and anonymous /tickets (expecting 401); with no LIVE_API_URL it prints SKIP and exits 0.

The code:

```
const base = (process.env.LIVE_API_URL || '').replace(/\/$/, '');
if (!base) { console.log('SKIP  LIVE_API_URL not set ...'); return; }
```

What's happening: Creates nothing: no tickets, users, or objects. Health may be 404 (correctly omitted from the API) or 200.

Why it matters: Safe post-deployment check that cannot incur writes.

PUTTING IT TOGETHER. The three scripts form the safety ladder: isolated full-stack smoke locally, local-only reset, and a read-only live probe that does nothing until deployment exists.

File: backend/services/api.js (legacy note) + backend/data/tickets.json

Byte 1: A tiny fetch wrapper predates the token-aware client

Builds on: None — starting point

In plain terms: This backend-side copy offers list/create/get/update/comment over localhost:3000 with no auth headers.

The code:

```
const API_BASE_URL = 'http://localhost:3000'
async function request(path, options = {}) {
  const response = await fetch(`${API_BASE_URL}${path}`, ...)
```

What's happening: It throws Error(data?.error?.message) on !ok. The frontend's src/services/api.js supersedes it with token providers, query builders, and upload support.

Why it matters: Kept for reference; new code should use the frontend token-aware client.

Byte 2: Seed data shows the ticket shape

Builds on: Byte 1

In plain terms: The JSON seed file is a small array of ticket rows with activity/comments/attachments arrays.

The code:

```
"ticketId": "…", "title": "…", "status": "Open", "createdBy": "customer-a", "activity": [...],
"comments": [...]
```

What's happening: loadTickets() normalizes any missing arrays, so minimal seeds still load. Reset restores [].

Why it matters: Reading one seed row teaches the whole document model.

PUTTING IT TOGETHER. The legacy service file and seed JSON are learning aids: minimal fetch patterns and a concrete ticket document, both superseded/normalized by the real client and loaders.

File: backend/deploy/functions.json + api-routes.json (+ cognito/execution-role/s3-cors companions)

Byte 1: One zip, seven functions, pinned runtime

Builds on: None — starting point

In plain terms: The deploy plan shares backend/dist/ticketing-handlers.zip across all tkt-* functions; only Handler differs, on nodejs22.x in ap-southeast-2.

The code:

```
{ "artifact": "backend/dist/ticketing-handlers.zip", "runtime": "nodejs22.x",
  "functions": [{ "name": "tkt-createTicket", "handler": "src/handlers/createTicket.handler",
"route": "POST /tickets" }, ...] }
```

What's happening: Env contract is TABLE_NAME=tkt-tickets + BUCKET_NAME, with documented aliases. Memory 512 MB, timeout 30 s. SHA recorded for artifact verification.

Why it matters: Sharing one artifact guarantees all routes deploy the same code.

Byte 2: API plan: JWT on everything except health

Builds on: Byte 1

In plain terms: All seven business routes require the Cognito JWT authorizer; /health is intentionally omitted from the API.

The code:

```
{ "method": "GET", "path": "/tickets/{ticketId}", "target": "tkt-getTicket", "auth": "JWT" },
{ "method": "GET", "path": "/health", "target": null, "auth": "NONE", "note": "OMIT from the
API..." }
```

What's happening: CORS allows only localhost:5173 with authorization/content-type headers. Invoke permissions are scoped per-function to the API's execution ARN.

Why it matters: Leaving health out avoids implying production readiness the handlers do not provide.

PUTTING IT TOGETHER. Deploy JSONs are unapplied plans (marked NOT APPLIED/CREATED): one shared zip for seven Node 22 functions plus a JWT-everywhere HTTP API with localhost CORS — the exact checklist for Stage 5/6 once approvals land.

7. KNOWLEDGE BYTES — Frontend foundation

File: frontend/package.json + vite.config.js + index.html + src/main.js + src/App.vue + .env.example

Byte 1: Vite + Vue + OIDC + Tailwind in six lines of config

Builds on: None — starting point

In plain terms: The build is Vite with the Vue plugin and Tailwind v4, pinned to port 5173 so CORS and Cognito redirect URIs stay stable.

The code:

```
plugins: [tailwindcss(), vue()],
server: { port: 5173, strictPort: true }
```

What's happening: Dependencies: vue, vue-router, oidc-client-ts (Cognito code flow), pinia (present though views use composables), tailwindcss. Tests run on node:test with no browser.

Why it matters: strictPort fails fast instead of silently moving to 5174 and breaking CORS/redirects.

Byte 2: Boot is three lines; env names are the switch

Builds on: Byte 1

In plain terms: main.js mounts the app with the router; .env.example shows that empty Cognito values mean local demo mode.

The code:

```
createApp(App).use(router).mount('#app')
// .env.example:
// VITE_API_URL=http://localhost:3000
// VITE_COGNITO_AUTHORITY=
// VITE_COGNITO_CLIENT_ID=
```

What's happening: App.vue is just <RouterView/>. index.html mounts #app and loads /src/main.js. isAuthConfigured() checks AUTHORITY+CLIENT_ID; when empty the app uses mock auth.

Why it matters: The presence or absence of two env vars flips the entire auth system.

PUTTING IT TOGETHER. Frontend foundation is minimal by design: Vite serves on a fixed port, main.js mounts a router-only App, and two env vars decide between local mock mode and Cognito — no other config needed to start.

File: frontend/src/auth/cognito.js (101 lines)

Byte 1: OIDC manager talks to Cognito Hosted UI

Builds on: None — starting point

In plain terms: A lazily-created UserManager performs the authorization-code flow against the Cognito authority.

The code:

```
manager = new UserManager({
  authority: cognitoAuthority.replace(/\/$/, ''),
  client_id: clientId, redirect_uri: redirectUri,
  response_type: 'code', scope,
  userStore: new WebStorageStateStore({ store: window.sessionStorage }) })
```

What's happening: Session lives in sessionStorage (per-tab). signInRedirect({prompt:'login'}) forces an explicit sign-in so Customer/Agent switching cannot silently reuse the wrong session.

Why it matters: Lazy creation plus isAuthConfigured() lets the app boot with no Cognito at all.

Byte 2: Role rule mirrors the backend exactly

Builds on: Byte 1

In plain terms: Agent means cognito:groups contains 'Agent'; everyone else is a Customer — same sentence as the backend.

The code:

```
export function roleFromProfile(profile = {}) {
  return groupsFromProfile(profile).includes('Agent') ? 'Agent' : 'Customer'
}
```

What's happening: groupsFromProfile tolerates array or single-string forms. signOutRedirect clears local OIDC state immediately instead of relying on Cognito end-session (which 400s on some setups).

Why it matters: Frontend and Lambda authorization can never disagree on who is an Agent.

PUTTING IT TOGETHER. cognito.js owns production identity: code-flow UserManager in sessionStorage, forced interactive login for role switching, and a backend-identical Agent rule — with safe no-op behavior when unconfigured.

File: frontend/src/auth/mock.js + src/auth/guard.js

Byte 1: Mock tokens live in sessionStorage with a test fallback

Builds on: None — starting point

In plain terms: Local tokens persist per-tab in browsers and in memory under node:test.

The code:

```
const STORAGE_KEY = 'ticketing.mockToken'
function webStorage() {
  try { if (typeof sessionStorage !== 'undefined') return sessionStorage }
  catch { /* unavailable (tests, SSR) */ }
  return null
}
```

What's happening: MOCK_USERS mirrors the backend fixtures (asserted in tests). readMockPayload only decodes for display — verification always happens server-side.

Why it matters: Display-decode vs verify is the discipline that keeps mock doors closed in prod.

Byte 2: Route guard is a pure function of route + auth state

Builds on: Byte 1

In plain terms: isAllowed decides allow/redirect without touching the browser, so it unit-tests with plain node.

The code:

```
if (requiresAuth && !authenticated) return { allowed: false, redirect: '/login' }
if (requiresAgent && !isAgent) return { allowed: false, redirect: '/' }
if (to.path === '/login' && authenticated) return { allowed: false, redirect: '/' }
```

What's happening: Public pages stay reachable in both modes; protected pages always need a session — there is no silent demo bypass. Authenticated users bounce off /login.

Why it matters: Pure logic means the router file stays thin and the rules stay tested.

PUTTING IT TOGETHER. Mock storage plus a pure guard give local auth its shape: per-tab fake sessions, display-only decoding, and testable allow/redirect decisions with no bypasses.

File: frontend/src/auth/useAuth.js (112 lines)

Byte 1: One composable unifies Cognito and mock sessions

Builds on: None — starting point

In plain terms: useAuth exposes authenticated/role/isAgent/email/displayName/userId the same way regardless of which auth mode is active.

The code:

```
const mockMode = computed(() => !configured.value)
const authenticated = computed(() =>
  configured.value ? !!oidcUser.value && !oidcUser.value.expired : !!mockUser.value)
```

What's happening: role reads roleFromProfile in Cognito mode or the mock fixture role locally; demoMode means unconfigured + logged-out. All state is module-level refs shared across components.

Why it matters: Components never branch on auth mode for display logic.

Byte 2: Session restore and role-switch hygiene

Builds on: Byte 1

In plain terms: On boot the composable revalidates any saved session; loginAs/clear paths keep storage and memory in sync.

The code:

```
const res = await fetch(`${API_URL}/auth/mock-me`, { headers: { Authorization: `Bearer ${token}`
} })
if (!res.ok) throw new Error('session expired')
// ... catch { clearMockToken(); mockUser.value = null }
```

What's happening: loginAs POSTs /auth/mock-login then stores token+user; logout clears both mock and OIDC state. Customer/Agent login pages clear the opposite session before starting a new redirect.

Why it matters: Stale or wrong-role sessions cannot linger across portal switches.

PUTTING IT TOGETHER. useAuth is the single identity source for the UI: mode-agnostic computed user properties, server-revalidated mock restore, and clean logout/switch semantics.

File: frontend/src/services/api.js (130 lines)

Byte 1: Token provider plugs in once, no import cycles

Builds on: None — starting point

In plain terms: The client asks a registered function for the current token on every call instead of importing auth state directly.

The code:

```
export function setAuthTokenProvider(fn) {
  tokenProvider = typeof fn === 'function' ? fn : null
}
async function authHeaders() {
  const token = await tokenProvider()
  return token ? { Authorization: `Bearer ${token}` } : {}
}
```

What's happening: Registered once at startup (see Dashboard). In local demo no provider means no header and the backend mock path applies; in prod the Cognito JWT is attached.

Why it matters: Decoupling avoids circular imports between api and auth modules.

Byte 2: Uniform errors plus a two-step upload

Builds on: Byte 1

In plain terms: Failed calls throw Error with status/code/path; uploads first fetch a presigned URL then PUT the bytes.

The code:

```
const serverMessage = data?.error?.message || data?.error
const err = new Error(message); err.status = response.status; err.code = data?.error?.code
// upload:
const { uploadUrl, key, mock } = await this.presignedUrl({ filename, contentType, fileSize })
```

What's happening: request() maps network failure to NETWORK_ERROR with a VITE_API_URL hint. toQuery drops empty filters. uploadAttachment enforces 5 MB client-side and appends ?ticketId= for local association.

Why it matters: Components get actionable errors and one-call uploads in both environments.

PUTTING IT TOGETHER. The API client is the UI's HTTP boundary: pluggable bearer tokens, envelope-aware errors, query building, and presign-then-PUT uploads that work against disk locally or S3 in production.

File: frontend/src/router/index.js (71 lines)

Byte 1: Seven routes, two workspaces, one catch-all

Builds on: None — starting point

In plain terms: Landing, login pickers, two dashboards, and the OIDC callback cover every screen; unknown paths return home.

The code:

```
{ path: '/tickets', component: Dashboard, meta: { requiresAuth: true } },
{ path: '/agent-dashboard', component: AgentDashboard, meta: { requiresAuth: true,
requiresAgent: true } },
{ path: '/callback', component: Callback, meta: { requiresAuth: false } },
{ path: '/:pathMatch(.*)*', redirect: '/' }
```

What's happening: CustomerLogin/AgentLogin are separate entry points so each portal can demand its role. currentAuthState reads mock expiry locally or getStoredUser() in Cognito mode.

Why it matters: URL structure itself enforces workspace separation.

Byte 2: Global guard plus workspace ping-pong prevention

Builds on: Byte 1

In plain terms: Every navigation consults isAllowed, then applies extra rules keeping Agents and Customers out of each other's home.

The code:

```
const decision = isAllowed(to, state)
if (state.authenticated && state.isAgent && to.path === '/tickets') return '/agent-dashboard'
if (state.authenticated && !state.isAgent && to.path === '/agent-dashboard') return '/tickets'
```

What's happening: Logged-in visitors to /login bounce to their workspace; Agents hitting requiresAgent-less customer pages (and vice versa) are rerouted instead of shown an error.

Why it matters: Users cannot sit in the wrong workspace even by typing URLs.

PUTTING IT TOGETHER. The router binds auth state to navigation: declarative per-route requirements plus a global guard that redirects strangers to login and keeps each role in its own dashboard.

8. KNOWLEDGE BYTES — Frontend views

File: frontend/src/views/Landing.vue + Login.vue

Byte 1: Landing sells the product; Login enforces role separation

Builds on: None — starting point

In plain terms: Landing is a marketing page with feature/workflow/security/architecture anchors; Login offers exactly two doors — Customer or Agent.

The code:

```
<RouterLink to="/customer-login" class="lp-role-card lp-customer-card">
<button class="role-option customer" @click="openCustomer">
function openCustomer() { router.push('/customer-login') }
```

What's happening: Login's security note states portal choice never grants permissions — Cognito group decides. Both pages link back home so users never get stuck.

Why it matters: Separate entry points preview the workspace split the router later enforces.

PUTTING IT TOGETHER. Landing and Login are the funnel: attractive overview, then an explicit Customer/Agent choice whose copy already warns that only the verified role matters.

File: frontend/src/views/CustomerLogin.vue + AgentLogin.vue + Callback.vue

Byte 1: Mirrored portals detect wrong-role sessions

Builds on: None — starting point

In plain terms: Each portal shows a session notice when you arrive signed in as the opposite role and offers a clean switch.

The code:

```
<div v-if="sessionNotice" class="session-notice">
  <strong>Agent session detected</strong>
async function loginCustomer() {
  if (auth.authenticated.value && auth.role.value !== 'Customer') {
    await clearStoredUser(); auth.setUserFromCallback(null) }
  await auth.login('Customer')
```

What's happening: Busy/error states disable the button while redirecting; unconfigured Cognito shows a setup error instead of hanging. Expected role is stored in sessionStorage (supportly_expected_role).

Why it matters: Users with two accounts can switch without clearing browser data manually.

Byte 2: Callback verifies role and recovers from mismatch

Builds on: Byte 1

In plain terms: After Cognito redirects back, Callback completes the code exchange, compares actual vs expected role, and routes accordingly.

The code:

```
return mgr.signinRedirectCallback()
// then:
function workspaceFor(role) {
  return role === 'Agent' ? '/agent-dashboard' : '/tickets' }
```

What's happening: Mismatch UI shows the signed-in role with three options: continue to the correct workspace, sign in with a different account, or go home. Failures show a back button instead of a blank page.

Why it matters: Wrong-portal sign-ins become a guided fix, not a dead end.

PUTTING IT TOGETHER. The sign-in trio handles real-world multi-account use: role-aware portals, opposite-session detection with clean switching, and a callback page that verifies roles and recovers gracefully from mismatches.

File: frontend/src/views/Dashboard.vue (customer workspace)

Byte 1: Customer home: tickets, search, create, favorites, drafts

Builds on: None — starting point

In plain terms: The customer dashboard lists your tickets with status/priority filters and text search, plus favorites and CSV export.

The code:

```
const tickets = ref([])
const search = ref('')
const statusFilter = ref('All statuses')
const favoriteIds = ref(readLocalJson(FAVORITES_KEY, []))
function mapTicket(ticket) { ... }
```

What's happening: mapTicket normalizes server rows for display; readLocalJson persists favorites (supportly_customer_favorites) and drafts (supportly_customer_ticket_draft) in localStorage. Token provider is registered here at startup.

Why it matters: Local niceties (drafts/favorites/export) never touch the API — they are pure browser features.

Byte 2: Create-with-attachment and follow-up flows

Builds on: Byte 1

In plain terms: New tickets support templates, draft restore, file attach via uploadAttachment, and one-click follow-ups.

The code:

```
function openCreateTicket() { ... }
function applyTemplate(template) { ... }
async uploadAttachment(file, ticketId) // presignedUrl then PUT
```

What's happening: Detail pane shows activity/comments with comment post and attachment download links. Errors surface apiError strings from the client's uniform error shape.

Why it matters: The dashboard exercises every customer API: list/get/create/comment/presign.

PUTTING IT TOGETHER. Dashboard.vue is the customer workspace: scoped ticket list with filters/search, draft- and favorite-aware creation with attachments, and detail with activity and comments — all through the token-aware client.

File: frontend/src/views/AgentDashboard.vue (agent workspace)

Byte 1: Queue metrics and workload views

Builds on: None — starting point

In plain terms: Agents see counts (open/in-progress/unassigned/urgent/resolved), resolution rate, my-tickets, and per-customer summaries.

The code:

```
const openCount = computed(() => tickets.value.filter(t => t.status === 'Open').length)
const myTickets = computed(() => tickets.value.filter(t => String(t.assignee||'').toLowerCase()
=== String(auth.email.value||'').toLowerCase()))
const resolutionRate = computed(() => totalCount.value ?
Math.round((resolvedCount.value/totalCount.value)*100) : 0)
```

What's happening: normalize() unifies legacy/new field names; normalizeComment does the same for comments. queueFilter/workspaceView slice the same array without extra fetches.

Why it matters: Managers get triage numbers; agents get 'my work' without new endpoints.

Byte 2: Agent actions: assign-to-me, status, reply

Builds on: Byte 1

In plain terms: One click claims a ticket; status/priority updates and replies call the Agent-only APIs.

The code:

```
async function assignToMe() { ... updateTicket({ assignee: auth.email.value }) }
async function updateTicket(changes) { ... }
async function openTicket(ticket) { ... refreshConversation(ticketId) }
```

What's happening: updateTicket surfaces 403 FORBIDDEN if a non-Agent ever reaches this screen (defense in depth with the router guard). Conversation refresh keeps comments current after posting.

Why it matters: The UI makes the Agent-only PUT discoverable while the backend remains the enforcer.

PUTTING IT TOGETHER. AgentDashboard is the support console: live queue stats, personal workload, customer summaries, and one-click assign/update/reply — all computed client-side over the scoped ticket list with Agent-gated mutations.

File: frontend/src/components/HelloWorld.vue + tests (guard.test.js, mock.test.js) + backend/tests/*

Byte 1: Sample component and pure-function tests

Builds on: None — starting point

In plain terms: HelloWorld is the default Vite sample (props + slots demo), not app UI; the frontend tests assert guard decisions and mock parity.

The code:

```
// guard.test.js: isAllowed({meta:{requiresAuth:true}}, {authenticated:false}) =>
{allowed:false, redirect:'/login'}
// mock.test.js: MOCK_USERS parity with backend/src/lib/mockUsers.js
```

What's happening: Backend suite (8 files, node:test): validation, auth/normalize, uploads, mock-auth escalation, http-auth middleware, handlers with fake db, local-server HTTP, single-table DAL. No test touches AWS.

Why it matters: Tests encode the security invariants (spoof rejection, mock rejection, 404 masking).

PUTTING IT TOGETHER. Samples, tests, and fixtures close the loop: an untouched Vite sample, frontend tests locking guard/mock behavior, and a backend suite proving validation, authz, and DAL logic with injected fakes.

9. Cross-file flows

Ticket creation (customer): Dashboard form → ticketApi.create + uploadAttachment (presignedUrl → PUT) → POST /tickets with attachmentKeys → validateCreateTicket + validateAttachmentKeys → DynamoDB Put (or JSON push locally) with CREATED activity → 201. Identity from token throughout; keys namespaced to caller.

Status/assign (agent): AgentDashboard assignToMe/updateTicket → PUT /tickets/{id} with Bearer JWT → attachUser/requireAgent → validateUpdateTicket + pickAssignee → activity append (STATUS_CHANGED/ASSIGNED) → Put → enriched ticket. Non-Agents get 403; anonymous 401.

Comments: detail pane → POST/GET /tickets/{id}/comments → canAccessComments (owner-or-Agent) else 404 → comment row (single-table COMMENT# or TicketComments) + COMMENT_ADDED activity. Chronological via Query ScanIndexForward.

Auth: unconfigured → mock-login fixtures + sessionStorage token + useAuth mockUser; configured → portal choice stores expected role → Cognito code flow → Callback verifies roleFromProfile vs expected → router guard routes to /tickets or /agent-dashboard. Backend independently derives role from cognito:groups; mocks rejected.

Attachments: validatePresignedRequest → plan/build key attachments/<sub>/… (or local/<sub>/…) → 501 without bucket locally mock URL with mock:true → PUT bytes → ticket references key; downloads via GetObject presign (300 s) or local file serve with owner-or-Agent check.

10. Important gotchas and edge cases

1. Mock tokens in production headers are rejected (claims.mock===true → null → 401); never 'fix' by accepting them. 2. Role comes only from cognito:groups — frontend-sent roles ignored; tests prove tampering fails. 3. 404 masks invisible tickets (never 403 for customers) — do not 'improve' to 403. 4. Group claim has 4 wire shapes — always use normalizeGroupClaim. 5. Owner match checks createdBy OR legacy ownerId, plus email fallback. 6. Update accepts assignee OR assigneeId (pickAssignee) — send either, handle both. 7. Attachment keys capped at 5, ≤512 chars, owner segment must equal caller sub, no '..'. 8. MIME allowlist (png/jpeg/gif/webp/pdf/txt) + 5 MB enforced twice (client + server). 9. Presign returns 501 until bucket approved — treat as config state, not an error to retry blindly. 10. Single-table handlers target tkt-tickets which may not exist yet — verify TABLE_NAME env and table approval before invoking. 11. CORS allows only localhost:5173 (+127.0.0.1); production origins need explicit update. 12. Changing Vite port breaks CORS + Cognito redirect URIs (strictPort is intentional). 13. Seeded bulk tickets carry BULKTEST markers — filter them in triage. 14. Health semantics differ: local 200 local-mock vs API plan omits /health (verify-live accepts 404 or 200). 15. Secrets: never commit .env.local / tokens; .env.example holds names only.

11. File inventory and coverage

Examined in full or substantively (44 files → 100 Knowledge Bytes): backend/package.json, server.js, aws-server.js, lambda.js, src/lib/auth.js, mockUsers.js, validation.js, response.js, uploads.js, tickets.js, ticketsSingleTable.js, 7 handlers, scripts/smoke-local.js, reset-local-data.js, verify-live.js, services/api.js, data/tickets.json, deploy/functions.json, api-routes.json (+ companions execution-role.json, s3-cors.json, cognito-new-pool.spec.json, admin-scoped-access.json surveyed), frontend/package.json, vite.config.js, index.html, src/main.js, App.vue, .env.example, auth/cognito.js, mock.js, guard.js, useAuth.js, services/api.js, router/index.js, views Landing/Login/CustomerLogin/AgentLogin/Callback/Dashboard/AgentDashboard/HelloWorld, tests (8 backend + 2 frontend surveyed), scripts/configure-frontend-aws.ps1, seed-ticket-batch.ps1.

Excluded (not application source): node_modules/, dist/, *.zip (ticketing-handlers, amplify-deploy, ticket-detail-fixed, kt-listTickets-fixed), .supportly-*-backup*/ + patch-backups/, *.before-* snapshots (ticketsSingleTable.js.before-*, Dashboard.vue.before-*, AgentDashboard.vue.before-*), package-lock.json, public icons/hero images, seeded-tickets-*.json dumps, *-smoke-response.json / lambda-*-event/response fixtures, .env.local/.env.production.local contents (secrets — purpose described only).

Counts: source files examined 44, files documented 44 (in 33 grouped sections), Knowledge Bytes produced 62, files excluded/non-source ~40, files not fully documented: Dashboard.vue + AgentDashboard.vue + ticketsSingleTable.js tail (large UI/markup sections summarized by structure scan - behavior verified via script-setup + tests; full line-by-line prose deferred to avoid clipping). No application source files were modified.